WFLOW-03: Alert Notification Basics

> **Series:** WFLOW — Workflows and Alert Notifications | **Notebook:** 3 of 10 | **Created:** January 2026 | **Last Updated:** 08/12/2026

Sending Notifications with Workflows

The most common workflow use case is sending alert notifications to Slack, Microsoft Teams, and email. This notebook covers setting up connections, configuring notification tasks, and best practices for effective alerting.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription

Requirement	Details
Permissions	`automation:workflows:write`, `automation:connections:write`
Prior Knowledge	**WFLOW-01** and **WFLOW-02**
External Access	Slack workspace admin or Teams channel access

1. Notification Architecture

How Workflow Notifications Work

```
Trigger (Detected Problem, Schedule, etc.)
            ↓
    Workflow Executes
            ↓
    Notification Task
            ↓
    Connection (credentials)
            ↓
    External Service (Slack, Teams, Email)
```

Components

Component	Purpose
Connection	Securely stores credentials (tokens, webhooks)
Notification Task	Sends message using connection
Message Template	Dynamic content using Jinja expressions

Supported Notification Channels

Channel	Task Type	Authentication
Slack	`dynatrace.slack:message`	OAuth App or Webhook
Microsoft Teams	`dynatrace.msteams:message`	Power Automate or Webhook (deprecated)

Channel	Task Type	Authentication
Email	`dynatrace.email:send`	SMTP or built-in
PagerDuty	`dynatrace.pagerduty:*`	Integration Key
ServiceNow	`dynatrace.servicenow:*`	OAuth or Basic Auth
Jira	`dynatrace.jira:*`	API Token
Custom Webhook	`dynatrace.http:request`	Bearer/Basic/Custom

2. Setting Up Connections

Connections store credentials separately from workflows for security and reusability.

Accessing Connections

1. Open **Settings** (gear icon)
2. Navigate to **Integration** → **Connections**
3. Or use direct URL: `https://<env>/ui/apps/dynatrace.hub/connections`

Creating a Connection

1. Click **+ Connection**
2. Select connection type (Slack, Teams, etc.)
3. Enter credentials
4. Name the connection (e.g., `slack-production-alerts`)
5. Save

Connection Best Practices

Practice	Description
Descriptive names	`slack-prod-oncall` not `slack1`
Environment separation	Different connections for prod/staging

Practice	Description
Least privilege	Minimum required scopes
Regular rotation	Update tokens periodically

3. Slack Notifications

Option 1: Slack App (Recommended)

Provides richer features: channels, DMs, reactions, threads.

Setup:
1. Create Slack App at https://api.slack.com/apps
2. Add OAuth scopes: `chat:write`, `chat:write.public`
3. Install to workspace
4. Copy Bot User OAuth Token
5. Create connection in Dynatrace with token

Option 2: Incoming Webhook (Simpler)

Single channel, no advanced features.

Setup:
1. Go to Slack channel settings → Integrations
2. Add **Incoming Webhook**
3. Copy webhook URL
4. Create connection with webhook URL

Basic Slack Message Task

```
name: send_slack_alert
type: dynatrace.slack:message
input:
  connection: slack-production
  channel: "#alerts-production"
  message: |
    :rotating_light: *Problem Detected*

    *Title:* {{ event()["title"] }}
    *Severity:* {{ event()["severity"] }}
    *Started:* {{ event()["start_time"] }}

    &lt;{{ event()["problem_url"] }}|View in Dynatrace&gt;
```

Slack Message with Blocks

For richer formatting:

```
input:
  connection: slack-production
  channel: "#alerts"
  blocks:
    - type: header
      text:
        type: plain_text
        text: "{{ ':red_circle:' if event()['severity'] == 'CRITICAL' else
':large_orange_circle:' }} {{ event()['severity'] }} Alert"
    - type: section
      fields:
        - type: mrkdwn
          text: "*Problem:*\n{{ event()['title'] }}"
        - type: mrkdwn
          text: "*Status:*\n{{ event()['status'] }}"
    - type: section
      fields:
        - type: mrkdwn
          text: "*Started:*\n{{ event()['start_time'] }}"
        - type: mrkdwn
          text: "*ID:*\n{{ event()['display_id'] }}"
    - type: actions
      elements:
        - type: button
          text:
            type: plain_text
            text: "View Problem"
          url: "{{ event()['problem_url'] }}"
```

4. Microsoft Teams Notifications

> **Warning — O365 Connectors Deprecated:** Microsoft retired Office 365 Connectors (Incoming Webhooks) in late 2024. Existing webhooks may continue to work temporarily, but **new O365 Connector creation is disabled**. Use one of these alternatives instead:

Method	Status	Notes
Power Automate Workflow	Recommended	Create a "When a Teams webhook request is received" flow in Power Automate; use the resulting URL as a `dynatrace.http:request` task

Method	Status	Notes
Teams Workflows Connector	Recommended	Available in new Teams client under channel **...** → **Workflows**
O365 Incoming Webhook	Deprecated	Legacy method shown below for reference only

Setting Up Teams Webhook (Legacy)

1. Open target Teams channel
2. Click **...** → **Connectors** (or **Workflows** in new Teams)
3. Add **Incoming Webhook**
4. Name it (e.g., "Dynatrace Alerts")
5. Copy the webhook URL
6. Create connection in Dynatrace

Basic Teams Message Task

```
name: send_teams_alert
type: dynatrace.msteams:message
input:
  connection: teams-production
  message: |
    **Problem Detected**

    **Title:** {{ event()["title"] }}
    **Severity:** {{ event()["severity"] }}
    **Started:** {{ event()["start_time"] }}

    [View in Dynatrace]({{ event()["problem_url"] }})
```

Teams Adaptive Card

For richer formatting:

```
input:
  connection: teams-production
  card:
    type: AdaptiveCard
    $schema: "http://adaptivecards.io/schemas/adaptive-card.json"
    version: "1.4"
    body:
      - type: TextBlock
        text: "{{ event()['severity'] }} Alert"
        size: Large
        weight: Bolder
        color: "{{ 'Attention' if event()['severity'] == 'CRITICAL' else
'Warning' }}"
      - type: FactSet
        facts:
          - title: "Problem"
            value: "{{ event()['title'] }}"
          - title: "Status"
            value: "{{ event()['status'] }}"
          - title: "Started"
            value: "{{ event()['start_time'] }}"
    actions:
      - type: Action.OpenUrl
        title: "View in Dynatrace"
        url: "{{ event()['problem_url'] }}"
```

5. Email Notifications

Email Options

Option	Setup	Best For
Built-in	No setup required	Simple notifications
Custom SMTP	Configure SMTP server	Corporate email servers
SendGrid/SES	API integration	High volume, templates

Basic Email Task

```
name: send_email_alert
type: dynatrace.email:send
input:
  to:
    - "oncall@company.com"
    - "platform-team@company.com"
  subject: "[{{ event()['severity'] }}] {{ event()['title'] }}"
  body: |
    A problem has been detected in your Dynatrace environment.

    Problem Details:
    ----------------
    Title: {{ event()["title"] }}
    Severity: {{ event()["severity"] }}
    Status: {{ event()["status"] }}
    Start Time: {{ event()["start_time"] }}
    Problem ID: {{ event()["display_id"] }}

    Affected Entities:
    {{ event()["affected_entity_ids"] | join(", ") }}

    Root Cause:
    {{ event()["root_cause_entity_id"] }}

    View this problem:
    {{ event()["problem_url"] }}
```

HTML Email

```
input:
  to: ["oncall@company.com"]
  subject: "[{{ event()['severity'] }}] {{ event()['title'] }}"
  contentType: "text/html"
  body: |
    &lt;html&gt;
    &lt;body style="font-family: Arial, sans-serif;"&gt;
      &lt;h2 style="color: {{ '#dc3545' if event()['severity'] == 'CRITICAL'
else '#ffc107' }};"&gt;
        {{ event()['severity'] }} Alert
      &lt;/h2&gt;
      &lt;table style="border-collapse: collapse;"&gt;
        &lt;tr&gt;&lt;td&gt;&lt;b&gt;Problem:&lt;/b&gt;&lt;/td&gt;&lt;td&gt;{{
event()['title'] }}&lt;/td&gt;&lt;/tr&gt;
        &lt;tr&gt;&lt;td&gt;&lt;b&gt;Status:&lt;/b&gt;&lt;/td&gt;&lt;td&gt;{{
event()['status'] }}&lt;/td&gt;&lt;/tr&gt;
        &lt;tr&gt;&lt;td&gt;&lt;b&gt;Started:&lt;/b&gt;&lt;/td&gt;&lt;td&gt;{{
event()['start_time'] }}&lt;/td&gt;&lt;/tr&gt;
      &lt;/table&gt;
      &lt;p&gt;&lt;a href="{{ event()['problem_url'] }}"&gt;View in
Dynatrace&lt;/a&gt;&lt;/p&gt;
    &lt;/body&gt;
    &lt;/html&gt;
```

6. Message Formatting

Jinja Expression Reference

Expression	Result	Use
`{{ event()["title"] }}`	Problem title	Main content
`{{ event()["severity"] }}`	CRITICAL/HIGH/MEDIUM/LOW	Color coding
`{{ event()["affected_entity_ids"] | join(", ") }}`	Comma-separated list	Show all affected
`{{ event()["start_time"] }}`	ISO timestamp	When started

Expression	Result	Use
`{{ event()["problem_url"] }}`	Full URL	Link to problem

Conditional Formatting

```
{% if event()["severity"] == "CRITICAL" %}
:red_circle: CRITICAL ALERT
{% elif event()["severity"] == "HIGH" %}
:large_orange_circle: HIGH ALERT
{% else %}
:large_yellow_circle: {{ event()["severity"] }} ALERT
{% endif %}
```

Severity Emoji Map

Severity	Slack Emoji	Teams Color
CRITICAL	`:red_circle:`	`Attention`
HIGH	`:large_orange_circle:`	`Warning`
MEDIUM	`:large_yellow_circle:`	`Accent`
LOW	`:large_blue_circle:`	`Good`

Inline Severity Mapping

```
{{ {"CRITICAL": ":red_circle:", "HIGH": ":large_orange_circle:", "MEDIUM":
":large_yellow_circle:", "LOW": ":large_blue_circle:"}.get(event()
["severity"], ":white_circle:") }}
```

7. Complete Alert Workflow Example

A production-ready workflow that sends alerts to Slack, Teams, and email.

Workflow Configuration

```
name: production-alert-notifications
description: Send alerts to all channels for production problems

trigger:
  type: davis-problem
  config:
    entityTagsMatch: all
    entityTags:
      - key: env
        value: prod

tasks:
  - name: slack_notification
    type: dynatrace.slack:message
    input:
      connection: slack-production
      channel: "#alerts-production"
      message: |
        {{ {"CRITICAL": ":red_circle:", "HIGH": ":large_orange_circle:",
"MEDIUM": ":large_yellow_circle:"}.get(event()["severity"], ":white_circle:")
}} *{{ event()["severity"] }} Problem*

        *{{ event()["title"] }}*

        • *Status:* {{ event()["status"] }}
        • *Started:* {{ event()["start_time"] }}
        • *ID:* {{ event()["display_id"] }}

        &lt;{{ event()["problem_url"] }}|:mag: View Problem&gt;

  - name: teams_notification
    type: dynatrace.msteams:message
    input:
      connection: teams-production
      message: |
        **{{ event()["severity"] }} Problem**

        **{{ event()["title"] }}**

        - **Status:** {{ event()["status"] }}
        - **Started:** {{ event()["start_time"] }}
        - **ID:** {{ event()["display_id"] }}

        [View Problem]({{ event()["problem_url"] }})
```

```
  - name: email_notification
    type: dynatrace.email:send
    input:
      to: ["platform-oncall@company.com"]
      subject: "[{{ event()['severity'] }}] {{ event()['title'] }}"
      body: |
        A {{ event()["severity"] }} problem has been detected.

        Problem: {{ event()["title"] }}
        Status: {{ event()["status"] }}
        Started: {{ event()["start_time"] }}

        View: {{ event()["problem_url"] }}
```

Task Execution

By default, tasks execute **in parallel**. All three notifications send simultaneously.

Monitor Your Notification Workflows

```
// Notification workflow executions
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript,
send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records —
filter on it, otherwise a
//                                        single execution is counted once as
RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                        (was task.error)  ·  duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| filter in(dt.automation_engine.action.app, {"dynatrace.email",
"dynatrace.slack", "dynatrace.servicenow"})
| summarize executions = count(), by:{dt.automation_engine.action.app,
dt.automation_engine.state}
| sort executions desc
```

```
// Notification task success rates
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| summarize {total = count(), succeeded = countIf(dt.automation_engine.state == "SUCCESS")}, by:{dt.automation_engine.action.function}
| fieldsAdd success_pct = round(succeeded * 100.0 / total, decimals: 1)
| sort total desc
| limit 20
```

```
// Failed notification tasks
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state == "ERROR"
| fields timestamp, dt.automation_engine.action.function, dt.automation_engine.task.name, dt.automation_engine.state_info
| sort timestamp desc
| limit 25
```

Next Steps

Now that you can send basic notifications, learn advanced patterns:

Recommended Path

1. **WFLOW-04: Advanced Notification Routing** - Conditional routing by severity, team, time
2. **WFLOW-05: PagerDuty & ServiceNow** - Incident management integration
3. **WFLOW-06: Custom Notification Templates** - Rich formatting and dynamic content

Key Takeaways

- **Connections** store credentials securely and are reusable
- **Slack** supports both OAuth apps and webhooks
- **Teams** requires Power Automate or Teams Workflows connector (O365 Incoming Webhooks are deprecated)
- **Email** can use built-in SMTP or custom servers
- **Jinja expressions** enable dynamic message content
- Tasks execute in **parallel** by default

Summary

In this notebook, you learned:

- Notification architecture: triggers → tasks → connections → external services
- How to set up connections for Slack, Teams, and email
- Basic and advanced Slack message formatting
- Microsoft Teams notifications with Adaptive Cards
- Plain text and HTML email notifications
- Jinja expressions for dynamic content
- A complete multi-channel alert workflow

References

- Notification actions umbrella (DT docs)

- Slack action (DT docs)
- Microsoft Teams action (DT docs)
- Email action (DT docs)
- PagerDuty action (DT docs)
- Workflow reference / Jinja expressions (DT docs)
- Alerting and notifications umbrella (DT docs)
